EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 3
AIM Create a dynamic To-Do List where users can add, mark as complete, and remove tasks using JavaScript & DOM.
Name: Ashish Ramesh Patel
Roll Number: 55
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Dr. Yogesh Pingle

Aim: Create a dynamic To-Do List where users can add, mark as complete, and remove tasks using JavaScript & DOM.

Theory

- A To-Do List application is a basic yet powerful example of a dynamic web application. It allows users to manage daily tasks by adding new tasks, marking tasks as completed, and removing tasks when they are no longer required. Such applications demonstrate the use of client-side scripting and real-time interaction without reloading the web page.
- The To-Do List application uses HTML for structure, CSS for presentation, and JavaScript for dynamic behavior. JavaScript interacts with the web page through the Document Object Model (DOM), making the page responsive to user actions.

A. **HTML (HyperText Markup Language)**

HTML is used to define the basic structure of the To-Do List application. It provides elements such as input fields, buttons, lists, and containers that form the user interface. In a To-Do List:

- Input elements are used to accept task details from the user
- Buttons are used to trigger actions like add or delete
- List elements are used to display tasks

Semantic HTML elements improve readability, accessibility, and maintainability of the application.

B. **CSS (Cascading Style Sheets)**

CSS is used to enhance the visual appearance of the To-Do List. It controls layout, colors, fonts, spacing, and alignment. CSS is also used to visually distinguish completed tasks, for example by applying a strike-through effect or changing text color. Proper styling improves user experience and clarity.

C. **JavaScript**

JavaScript is a client-side scripting language used to make web pages interactive. In a dynamic To-Do List application, JavaScript performs the following roles:

Captures user input

Handles user events such as button clicks

Updates the task list dynamically

Applies logic for task completion and deletion

JavaScript enables changes to occur instantly without refreshing the page.

D. **Document Object Model (DOM)**

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree-like structure of objects. Each HTML element becomes a node in the DOM. JavaScript can access and manipulate these nodes to dynamically change the content and structure of a web page.

In a To-Do List application, DOM manipulation is used to:

- Create new task elements dynamically
- Insert tasks into the list
- Modify task appearance when marked as complete

- Remove tasks from the list

E. **Event Handling**

Events are actions performed by the user such as clicking a button, typing text, or selecting an option. JavaScript uses event listeners to detect and respond to these actions.

In the To-Do List application:

Clicking the “Add” button triggers task creation

Clicking a checkbox or task marks it as completed

Clicking a delete button removes the task

Event handling is essential for creating interactive applications.

F. **Dynamic Task Management**

The dynamic nature of the To-Do List means that tasks are managed at runtime. Tasks are added, updated, or removed directly in the DOM.

This allows:

- Immediate visual feedback
- Efficient task handling
- Improved performance compared to page reloads
- Each task is treated as an independent DOM element, making management simple and scalable.

G. **Task Completion Feature**

Marking a task as complete is an important feature of a To-Do List. This is achieved by updating the task’s state using JavaScript and modifying its visual style using CSS. Completed tasks help users track progress and organize their work efficiently.

H. **Task Removal Feature**

The task removal feature allows users to delete unwanted or completed tasks. JavaScript removes the corresponding DOM element from the task list. This operation demonstrates effective DOM manipulation and memory management on the client side.

Procedure :

1. Create HTML structure:
```
<input id="task" type="text">
<button onclick="addTask()">Add</button>
<ul id="list"></ul>
```

2. Write JavaScript:
```
<script>
function addTask() {
  let task = document.getElementById('task').value;
  let li = document.createElement('li');
  li.innerText = task;
  li.onclick = () => li.remove();
  document.getElementById('list').appendChild(li);
}
</script>
```

3. Save and run in browser.

Code :

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Dynamic To-DO List</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
   <div class="conatiner">
      <h2>To-Do List</h2>
      <div class="input-box">
         <input type="text" id="taskInput" placeholder="Enter a task">
         <button onclick="addTask()">ADD</button>
      </div>
      <ul id="taskList"></ul>
   </div>

<script src="script.js"></script>
</body>
</html>
```

Style.css

```
body{
   font-size: Arial,sans-serif;
background: #f4f4f4;
   display: flex;     justify-
content: center;     align-
items: center;     height:
100vh;
}
.todo-container
{
   background: white;     padding:
20px;     width: 300px;     border-
radius: 8px;     box-shadow: 0 0 10px
rgba(0,0,0,0.1);
```

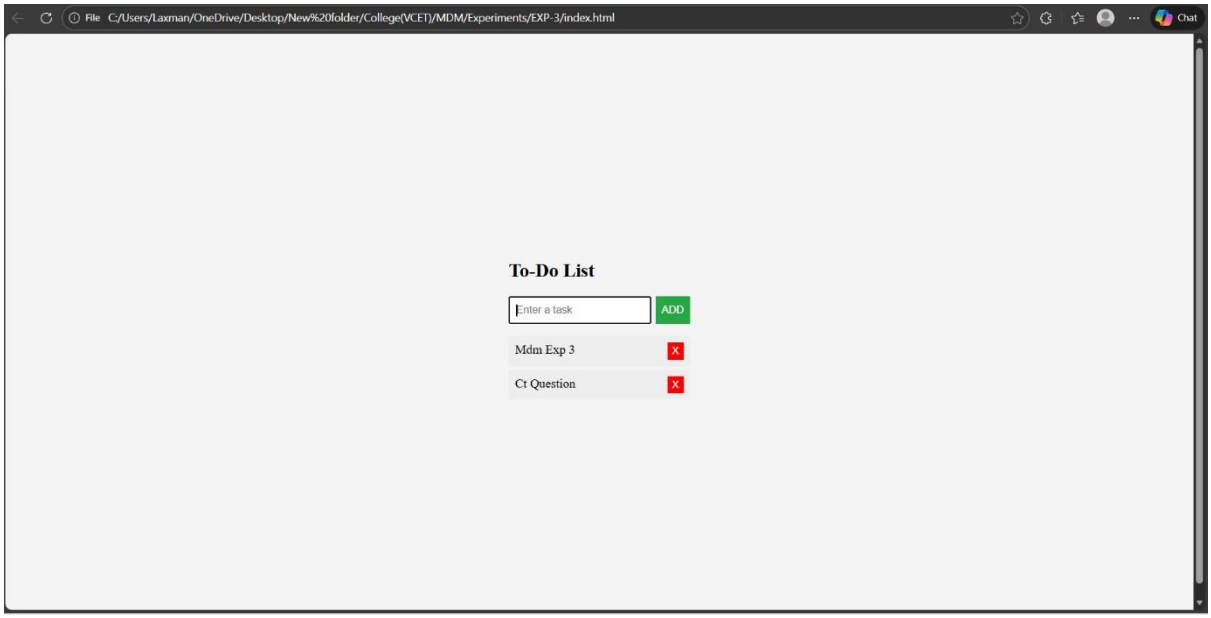
```
}
.input-box
{
    display: flex;
   gap:5px;
}
input
{
flex: 1;
    padding: 8px;
}
button
{
    padding: 8px;
background: #28a745;
    color: white;
border: none;
cursor: pointer;
}
button:hover
{
    background: #218838;
}
ul
{
    list-style:none ;
padding: 0;     margin-
top:15px;
}
li
{
   display: flex;
    justify-content: space-between;
padding: 8px;     background:
#eee;     margin-bottom:5px;
    cursor: pointer;
}
li.completed
{
    text-decoration: line-through;
    color: gray;
}
.delete
```

```
{
   background: red;
color:white;
border: none;
padding: 3px 6px;
cursor: pointer;
}
```

Script.js

```
function addTask(){    const taskInput
=document.getElementById("taskInput");    const taskText
=taskInput.value.trim();
  if(taskText===="")
  {
      alert("Please enter a task");
   }
   const li=document.createElement("li");
li.textContent=taskText;

   li.addEventListener("click",function()
   {
      li.classList.toggle("completed");
   });

   const deleteBtn=document.createElement("button");
deleteBtn.textContent="X";
deleteBtn.className="delete";

   deleteBtn.addEventListener("click",function(e)
   {
      e.stopPropagation();
li.remove();
   });

  li.appendChild(deleteBtn);
document.getElementById("taskList").appendChild(li);
  taskInput.value="";
}
```

Output



Conclusion:

The dynamic To-Do List was successfully implemented using JavaScript and DOM manipulation. Users can **add new tasks**, **mark tasks as complete**, and **delete tasks** in real-time without page reload. The project demonstrates the use of event handling, array management, and dynamic HTML updates, providing practical understanding of interactive web applications**.**

Questions:

1.What is an event listener? Give an example.

Ans:
Definition: An event listener is a procedure in JavaScript that waits for an event (like click, keypress, hover) to occur on an element and executes a function in response.
Example:
Javascript:

```
document.getElementById("btn").addEventListener("click", function() {
   alert("Button clicked!");
});
```

Key Points:

- Used to handle user interactions dynamically.
- Can be attached to HTML elements.
- Supports multiple event types (click, input, mouseover, etc.).

2. What is the difference between innerHTML and textContent?

Ans:

Aspect	innerHTML	textContent
Definition	Returns/sets HTML content inside an element	Returns/sets only text content inside element
HTML Tags	Parses and renders HTML tags	Treats tags as plain text
Use Case	When you need to insert HTML markup dynamically	When you only need raw text
Example	element.innerHTML = "Hello"; → bold text	element.textContent = "Hello"; → shows Hello literally

3.What is the difference between synchronous and asynchronous JavaScript in this context?

Ans:

Aspect	Synchronous JavaScript	Asynchronous JavaScript
Definition	Executes code line by line, blocking further tasks	Executes tasks without blocking main thread
Execution Flow	Next statement waits until current finishes	Allows other operations while task completes
Example	alert("Hello"); console.log("World"); → waits	setTimeout(()=>console.log("World"),1000);
Use Case	Simple, sequential tasks	API calls, timers, event handling, promises